

Introdução ao Front-end

Objetivo da Aula

O objetivo principal desta aula é fornecer uma visão abrangente sobre front-end e o back-end e conhecer quais são as suas principais funções, além da forma como eles trabalham juntos na construção de aplicações flexíveis e escaláveis.

Apresentação

Seja bem-vindo à aula de Introdução ao Front-End. Nesta jornada de aprendizado, o objetivo principal desta aula é fornecer uma visão abrangente sobre o front-end e o back-end e compreender quais são as suas principais funções, além de como eles trabalham juntos para criar aplicações flexíveis e escaláveis.

Inicialmente, você aprenderá os principais elementos do front-end em aplicações web, que é a camada responsável por apresentar a interface do usuário e proporcionar a interação com a aplicação, utilizando tecnologias como HTML, CSS e JavaScript.

Em seguida, você irá compreender como ocorre a comunicação cliente/servidor, os tipos de requisições que podem ser enviadas ao servidor e os tipos de respostas que o servidor pode retornar ao cliente, através do protocolo HTTP.

Por fim, compreenderá a evolução das aplicações web, desde as aplicações estáticas e dinâmicas até as aplicações web, com o front-end separado do back-end.

1. Elementos do Front-End em Aplicações Web

O front-end é a parte que lida com a interface e a experiência do usuário. É a camada de apresentação e interação que o usuário final vê e com a qual interage em um site ou aplicativo. O objetivo principal do front-end é fornecer uma interface agradável, intuitiva e responsiva para os usuários (Uzayr, 2022).

No desenvolvimento de aplicações web, as tecnologias Hypertext Markup Language (HTML), Cascading Style Sheets (CSS) e JavaScript, que são todas integradas e interpretadas no navegador (Firefox, Chrome, Opera, entre outros), são utilizadas no front-end. O navegador é uma aplicação que desempenha um papel crucial, ao interpretar o código HTML, aplicar estilos CSS, além de interpretar e executar scripts em JavaScript, permitindo que usuários interajam e visualizem a interface de forma intuitiva e amigável.

- **HTML** é a linguagem de marcação que define a estrutura e a organização do conteúdo de uma página web. Com o HTML, podemos criar elementos como cabeçalhos, parágrafos, listas, imagens e links;
- **CSS** é uma linguagem de estilização que permite controlar o layout, as cores, as fontes e outros aspectos visuais de um site. Com o CSS, podemos criar um design atraente e personalizado para melhorar a aparência do front-end;
- **JavaScript** é uma linguagem de programação que permite adicionar interatividade e funcionalidades avançadas ao front-end. Com o JavaScript, podemos criar animações, validar formulários, fazer requisições assíncronas e muito mais.

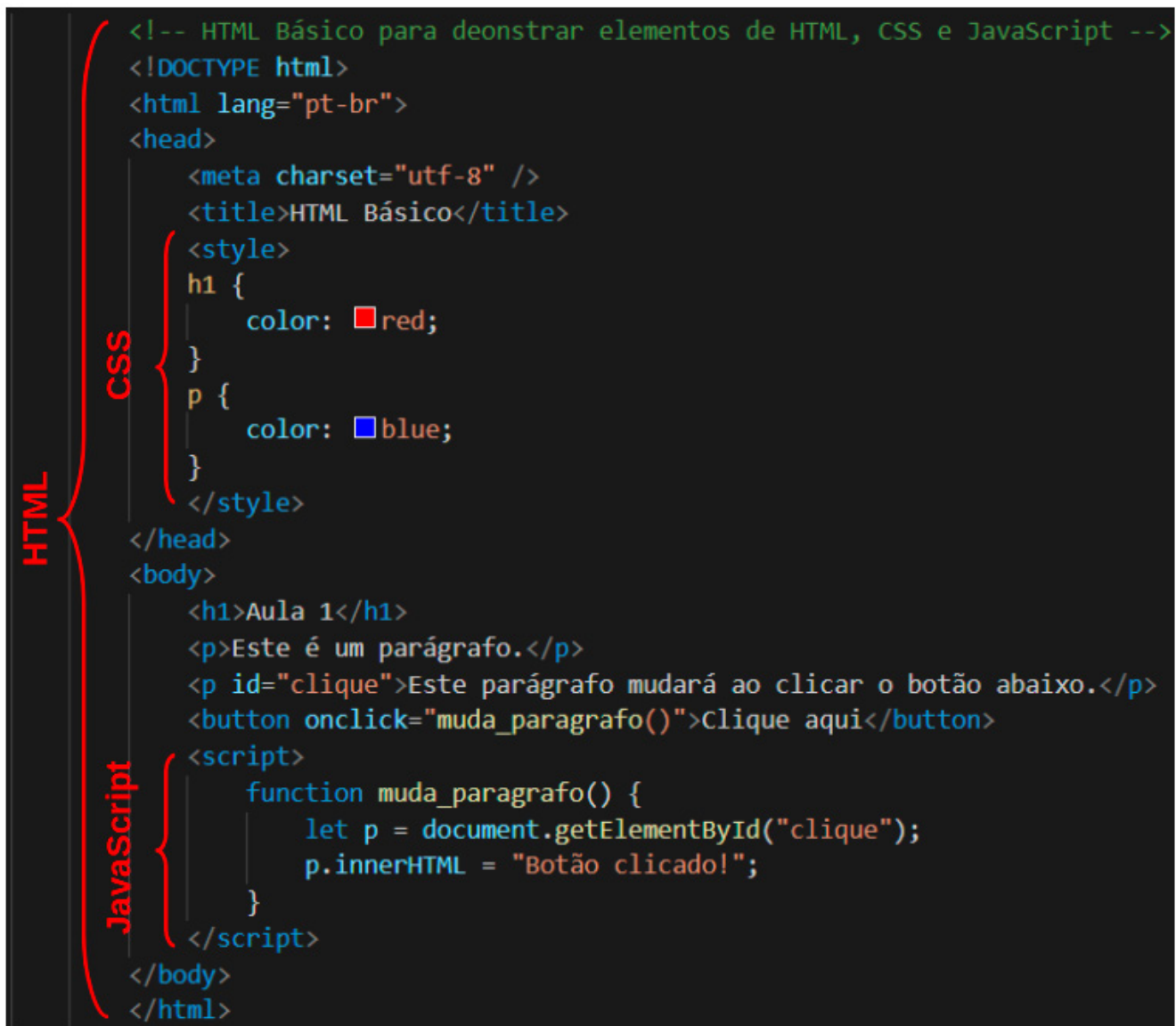


Destaque

Não confundir JavaScript com Java, pois são linguagens diferentes.

A Figura 1 apresenta código-fonte que ilustra a utilização de elementos HTML, CSS e JavaScript para a construção da interface. Conforme o código da página HTML da Figura 1, a estilização dos elementos HTML foi especificada no bloco de código entre as marcações `<style>` e `</style>`. Nessa especificação, todo cabeçalho definido com a marcação `<h1>` é estilizado com a cor vermelha, e todo parágrafo definido com a marcação `<p>` é estilizado com a cor azul.

Figura 1: Código de uma página HTML construído para demonstrar a utilização de elementos HTML, CSS e JavaScript



```

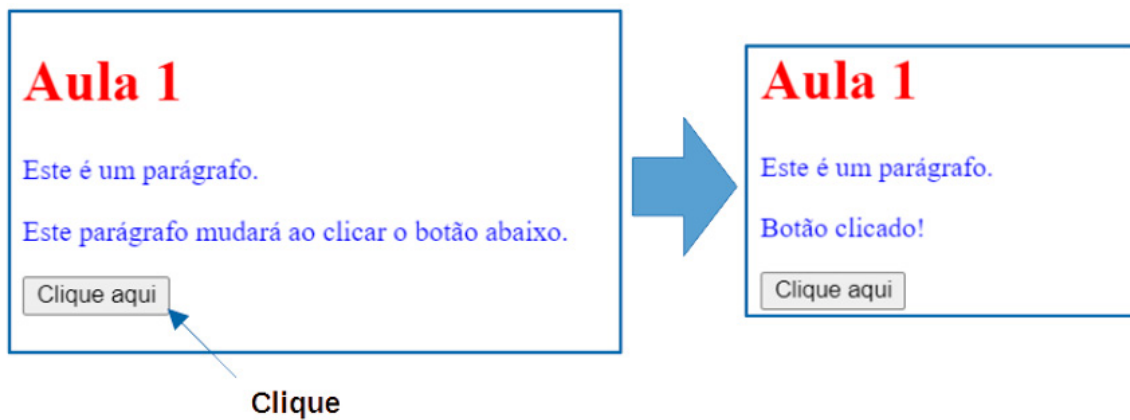
<!-- HTML Básico para deonstrar elementos de HTML, CSS e JavaScript -->
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8" />
  <title>HTML Básico</title>
  <style>
    h1 {
      color: ■ red;
    }
    p {
      color: ■ blue;
    }
  </style>
</head>
<body>
  <h1>Aula 1</h1>
  <p>Este é um parágrafo.</p>
  <p id="clique">Este parágrafo mudará ao clicar o botão abaixo.</p>
  <button onclick="muda_paragrafo()">Clique aqui</button>
  <script>
    function muda_paragrafo() {
      let p = document.getElementById("clique");
      p.innerHTML = "Botão clicado!";
    }
  </script>
</body>
</html>
  
```

O código é apresentado em um editor de texto com fundo escuro. À esquerda, há três corchetes verticais coloridos que agrupam o código: um corchete verde para o HTML, um corchete laranja para o CSS e um corchete amarelo para o JavaScript. O código em si está colorido por sintaxe, com tags HTML em verde, CSS em laranja e JavaScript em amarelo.

Fonte: Elaboração própria.

Ainda, de acordo com o código da Figura 1, a linguagem JavaScript permite que a página HTML seja dinâmica, ao invés de estática. O trecho de código especificado entre as marcações `<script>` e `</script>` declara uma função nomeada `muda_paragrafo()`. Essa função é responsável por mudar o conteúdo interno da marcação `<p>`, identificada com o `id="clique"`, para `"Botão clicado!"`. De forma dinâmica, ao realizarmos um evento de clique no botão da página, o evento `onclick` realiza a chamada da função `muda_paragrafo()`, mudando o conteúdo do segundo parágrafo, conforme ilustra a Figura 2.

Figura 2: Interface resultante do código da Figura 1. A esquerda ilustra a página HTML inicial e a direita o resultado após a interação de clique

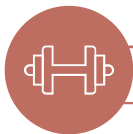


Fonte: Elaboração própria.



Você Sabia?

Em 1995, o JavaScript foi criado em apenas 10 dias pelo engenheiro da Netscape, Brendan Eich, com o intuito de melhorar a interatividade das páginas da web. Inicialmente, a linguagem era chamada de LiveScript, mas, posteriormente, foi renomeada para JavaScript, para capitalizar a popularidade da linguagem Java na época. Desde então, o JavaScript se tornou uma das linguagens de programação mais utilizadas no mundo, impulsionando a interatividade e a dinâmica das aplicações web modernas.



Para Praticar

Aprofundar os conhecimentos e praticar exemplos de HTML, CSS e JavaScript é de crucial importância para se tornar um bom desenvolvedor front-end. As documentações e os exemplos de HTML, CSS e JavaScript podem ser consultados em <https://www.w3schools.com> (acesso em 18 set. 2023.)

2. Arquitetura de Aplicações Web Cliente/Servidor

A arquitetura de aplicações web cliente/servidor é um modelo que descreve a distribuição de responsabilidades e a comunicação entre o cliente (navegador) e o servidor (Uzayr, 2022).

Essa comunicação entre o cliente e o servidor ocorre através do Protocolo de Transferência de Hipertexto (HTTP), do inglês Hypertext Transfer Protocol. Esse protocolo é utilizado para a transferência de dados entre o cliente (navegador) e o servidor web. Ele define a forma como as solicitações e respostas devem ser formatadas, transmitidas e processadas durante as interações entre o cliente e o servidor.

Com base em MDN (2023b), você aprenderá, na Seção 2.1., o protocolo de comunicação HTTP requisição-resposta, além dos tipos de requisições que o cliente pode enviar ao servidor e os tipos de respostas que o servidor pode responder ao cliente.

2.1. Protocolo de Comunicação HTTP

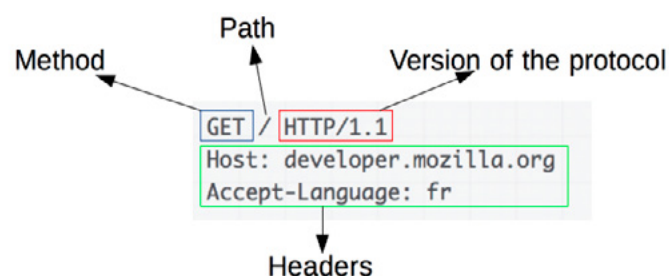
A versão de protocolo HTTP mais utilizada é a versão HTTP/1.1, baseada em uma comunicação requisição-resposta, em que o cliente realiza uma requisição e o servidor responde com uma ou mais respostas.

A seguir, você aprenderá os principais tipos de requisições HTTP que um cliente pode realizar, além dos principais tipos de resposta do servidor.

2.1.1. Requisições HTTP

De modo geral, as requisições HTTP são compostas de um método (method), um caminho (path), também conhecido como recurso ou endpoint, a versão do protocolo HTTP, os cabeçalhos (headers) e, opcionalmente, um corpo de dados a ser enviado para o servidor. A Figura 3 ilustra uma requisição com o método GET ao caminho / do host developer.mozilla.org.

Figura 3: Mensagem de requisição HTTP ao host developer.mozilla.org



Fonte: Mozilla. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: 1 jul. 2023.

Um método HTTP, geralmente, é um verbo como GET, POST, PUT, PATCH e DELETE ou um substantivo como OPTIONS ou HEAD. Esses métodos definem qual operação o cliente

deseja fazer no servidor, como solicitar um recurso do servidor (GET) ou enviar dados ao servidor (POST). Os principais métodos HTTP são descritos no Quadro 1.

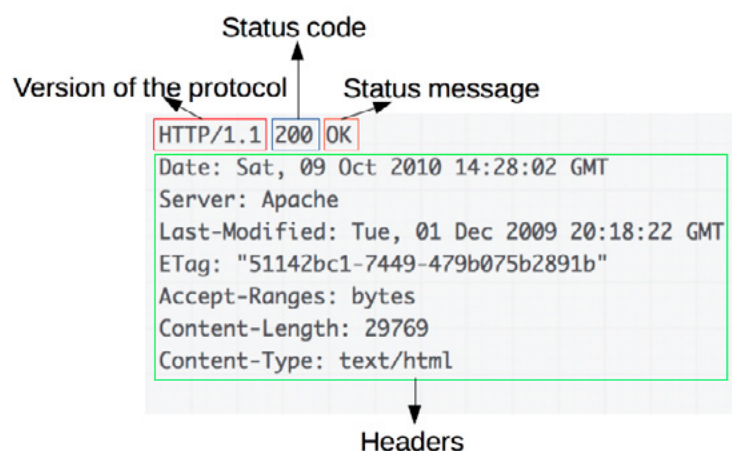
Quadro 1: Principais métodos HTTP utilizados para interações cliente/servidor

Método	Descrição
GET	Solicita um recurso específico identificado pelo endpoint. O método GET é seguro e não deve ter efeitos colaterais.
POST	Envia dados ao servidor para processamento. É, geralmente, usado para criar recursos ou enviar formulários. Pode ter efeitos colaterais no servidor.
PUT	Substitui completamente um recurso existente ou cria um recurso novo se não existir.
PATCH	Atualiza parcialmente um recurso existente, com as alterações enviadas no corpo da solicitação.
DELETE	Remove um recurso específico identificado pelo endpoint.
HEAD	Realiza uma solicitação semelhante ao GET, mas retorna apenas os cabeçalhos da resposta, sem o corpo da resposta.
OPTIONS	Obtém as opções de comunicação disponíveis para o recurso identificado pelo endpoint.

2.1.2. Respostas HTTP

As respostas HTTP são compostas da versão do protocolo HTTP utilizado, do código de estado (status code), indicando se a requisição foi bem-sucedida, do estado da mensagem (status message), descrevendo uma mensagem referente ao código de estado, além de cabeçalhos (headers) e, opcionalmente, um corpo de dados, assim como nas requisições. A Figura 4 ilustra uma resposta obtida do host developer.mozilla.org, com um código de estado 200 (OK), sob o protocolo HTTP/1.1, cujo tipo de conteúdo é text/html, ou seja, o corpo da mensagem tem o conteúdo de uma página HTML para ser renderizado no navegador do cliente.

Figura 4: Mensagem de resposta HTTP, obtida do host developer.mozilla.org



Fonte: Mozilla. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: 1 jul. 2023.

Um código de estado indica se uma requisição foi bem concluída ou não, sendo os tipos de respostas agrupados em cinco classes, conforme o Quadro 2.

Quadro 2: Classes de código de status de respostas HTTP

Códigos de status	Descrição
100-199	Respostas informativas, indicando que o servidor recebeu a solicitação e que o processo continua. Essas respostas são usadas para fins de comunicação e não afetam diretamente o processamento da solicitação.
200-299	Respostas de sucesso, indicando que a solicitação foi recebida, compreendida e processada com êxito pelo servidor. A resposta mais comum é o código 200 (OK), indicando que a solicitação foi bem-sucedida.
300-399	Respostas de redirecionamento, indicando que a solicitação requer ações adicionais para ser concluída. O cliente, geralmente, precisa seguir redirecionamentos para obter o recurso solicitado.
400-499	Respostas de erro do cliente, indicando que ocorreu um erro na solicitação feita pelo cliente. Isso pode ser devido a erros de sintaxe, autenticação, permissões insuficientes, entre outros.
500-599	Respostas de erro do servidor, indicando que ocorreu um erro no servidor ao processar a solicitação. Isso pode ser devido a problemas internos do servidor, tempo de resposta esgotado, entre outros.



Destaque

As respostas HTTP mais comuns e utilizadas são: i. 200 (OK), para confirmar que a requisição foi realizada com sucesso; ii. 400 (Bad Request), para indicar que a requisição do usuário contém erros; iii. 401 (Unauthorized), para indicar que o usuário necessita de autenticação para ter acesso a determinado recurso solicitado; iv. 404 (Not Found), indicando que o recurso solicitado não foi encontrado; e v. 500 (Internal Server Error), quando erros ocorrem no servidor ao realizar uma requisição.

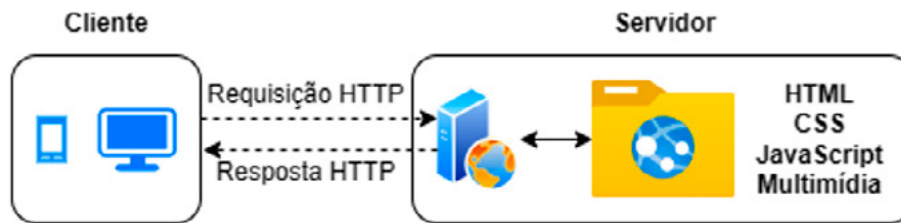
3. Modelos de Aplicações Web

As aplicações web evoluíram, ao longo do tempo, passando de aplicações estáticas para dinâmicas (MDN, 2023a), e, mais recentemente, para aplicações (ou interfaces) front-end separadas de aplicações (ou serviços) back-end.

Inicialmente, as aplicações web eram **estáticas**, conforme a Figura 5. Nesse modelo, o servidor é responsável por fornecer um conjunto fixo de arquivos HTML, CSS e JavaScript a serem processados e renderizados no navegador do usuário. As páginas exibidas no

navegador não mudam com base nas interações do usuário ou nos dados do servidor. O servidor simplesmente envia os arquivos estáticos ao cliente, que os exibe ao usuário final.

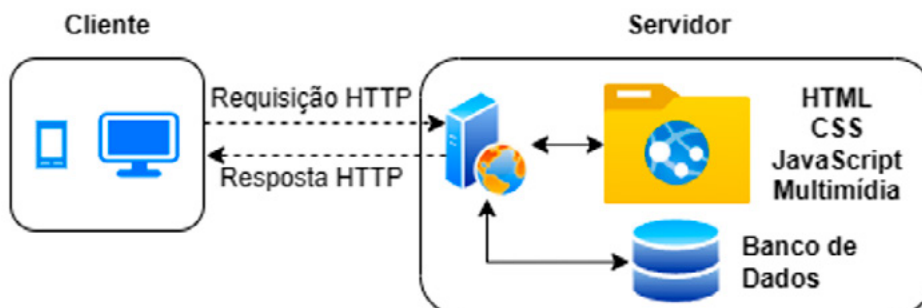
Figura 5: Modelo de aplicação web estática



Fonte: Elaboração própria.

Com o avanço das tecnologias e a demanda por interatividade, as aplicações web **dinâmicas** surgiram (veja a Figura 6). Nesse modelo, o servidor utiliza uma linguagem de programação, como PHP, Python, Ruby ou JavaScript (com Node.js), para manter informações e dados sensíveis do cliente seguros, em sessões do servidor, bem como processar as solicitações do cliente, acessar bancos de dados e gerar conteúdo personalizado ao cliente.

Figura 6: Modelo de aplicação web dinâmica



Fonte: Elaboração própria.

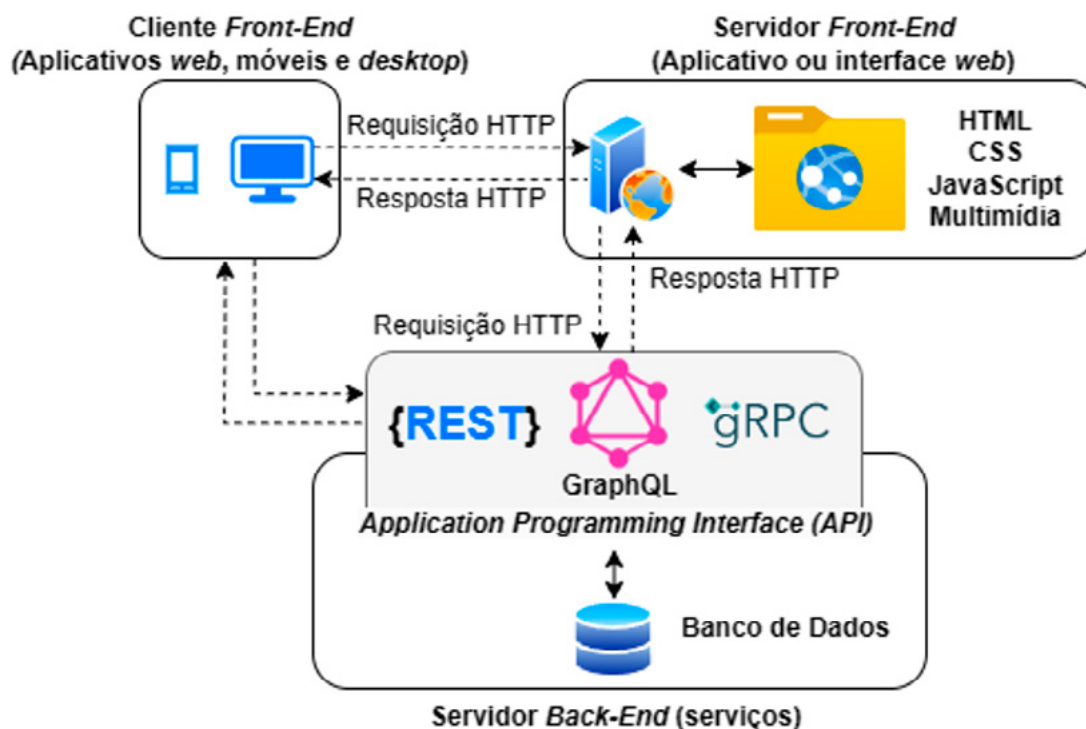


Destaque

A arquitetura mais comum de aplicações dinâmicas é o Model-View-Controller (MVC), sendo o modelo (model) responsável pela representação dos dados e das regras de negócio da aplicação, a visão (view) responsável pela apresentação da interface ao usuário e o controlador (Controller) responsável por gerenciar e controlar a interação do usuário com a aplicação.

Ao longo do tempo, a interface do usuário ganhou mais protagonismo e responsabilidades da aplicação, o que, antes, era exclusivamente do servidor. Na separação de responsabilidades, as aplicações web passaram a ser isoladas e desenvolvidas em duas partes independentes, conforme a Figura 7: o **front-end**, que trata da interface e da interatividade do usuário no lado do **cliente**; e **back-end**, que trata da lógica de negócios e manipulação de dados no lado do **servidor**.

Figura 7: Modelo de aplicação web dinâmica com front-end e back-end



Fonte: Elaboração própria.

Conforme a Figura 7, a separação do front-end e do back-end favorece a construção de uma abordagem arquitetural moderna, que oferece maior flexibilidade, escalabilidade, manutenção e reutilização de código, permitindo que diferentes interfaces do usuário (como aplicativos web, móveis e desktop) utilizem os mesmos serviços do back-end para envio e consumo de dados.



Destaque

Com a separação do front-end e do back-end, a arquitetura MVC também sofreu alterações, separando a visão, no front-end, do modelo e do controlador no back-end.

As APIs mais conhecidas e utilizadas no back-end são REST (Fielding, 2000), GraphQL¹ (2022) e gRPC² (2023).



Destaque

Os métodos HTTP GET, POST, PUT, PATCH e DELETE são amplamente utilizados por APIs REST, cujos verbos (GET, POST e outros) indicam uma ação a ser realizada no back-end.



Para Refletir

Sobre a importância de toda essa evolução no desenvolvimento de tecnologias web, como os conceitos apresentados podem ser aplicados em projetos reais e contribuir para o desenvolvimento de aplicações mais eficientes e interativas?

Considerações Finais da Aula

Ao final desta aula, você já é capaz de entender o que é front-end e qual é o seu papel na construção de aplicações web.

Aprendemos que o front-end é a parte do desenvolvimento responsável por criar uma interface em que o usuário possa interagir com a aplicação – seja essa interface web, móvel ou desktop. Em aplicações web, as interfaces são construídas utilizando HTML para estruturar o documento da página, CSS para estilizar visualmente a página e JavaScript para adicionar interatividade e funcionalidades avançadas.

Exploramos a arquitetura de aplicações web cliente/servidor, que fornece um modelo de comunicação entre o cliente e o servidor através de protocolos HTTP. Por meio dessa arquitetura, as aplicações web evoluíram de aplicações estáticas para dinâmicas, até a separação do front-end e do back-end, permitindo maior flexibilidade, reutilização de código e escalabilidade no seu desenvolvimento.

Material Complementar



Documentações e exemplos de HTML, CSS e JavaScript

2023, W3Schools.

Para impulsionar os seus conhecimentos em HTML, CSS e JavaScript, não deixe de explorar o W3Schools e praticar com os exemplos fornecidos. Aproveite essa valiosa ferramenta para se tornar um desenvolvedor web mais completo e confiante em suas habilidades.

Link para acesso: <https://www.w3schools.com> (acesso em 18 set. 2023.)

Referências

GRAPHQL. Disponível em: <https://graphql.org>. Acesso em: 1 jul. 2023.

GRPC. Disponível em: <https://grpc.io>. Acesso em: 1 jul. 2023.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. Irvine: University of California, 2000. ISBN 978-0-599-87118-2.

MDN contributors. **Introdução ao lado servidor**. Mdn Web Docs, 2023a. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn/Server-side/First_steps/Introduction. Acesso em: 23 jul. 2023.

MDN contributors. **Uma visão geral do HTTP**. Mdn Web Docs, 2023b. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: 12 dez. 2022.

UZAYR, Sufyan bin. **Frontend Development: The Ultimate Guide**. Boca Raton, Flórida: CRC Press, 2022.